

数据库实验报告（第七周）

班 级：3班

姓 名：梁力航

学 号：23336128

一、实验目的

- 熟悉SQL的数据控制操作
- 能够使用SQL语句对用户进行授予和收回权限

二、实验内容

- 使用GRANT语句对用户授权，对单个用户和多个用户授权，或使用保留字PUBLIC对所有用户授权。对不同的操作对象包括数据库、视图、基本表等进行不同权限的授权。
- 使用WITH GRANT OPTION子句授予用户传播该权限的权利。
- 在授权时发生循环授权，考察DBS能否发现这个错误。如果不能，结合取消权限操作，查看DBS对循环授权的控制。
- 使用REVOKE子句收回授权，查看取消授权的级联反应。

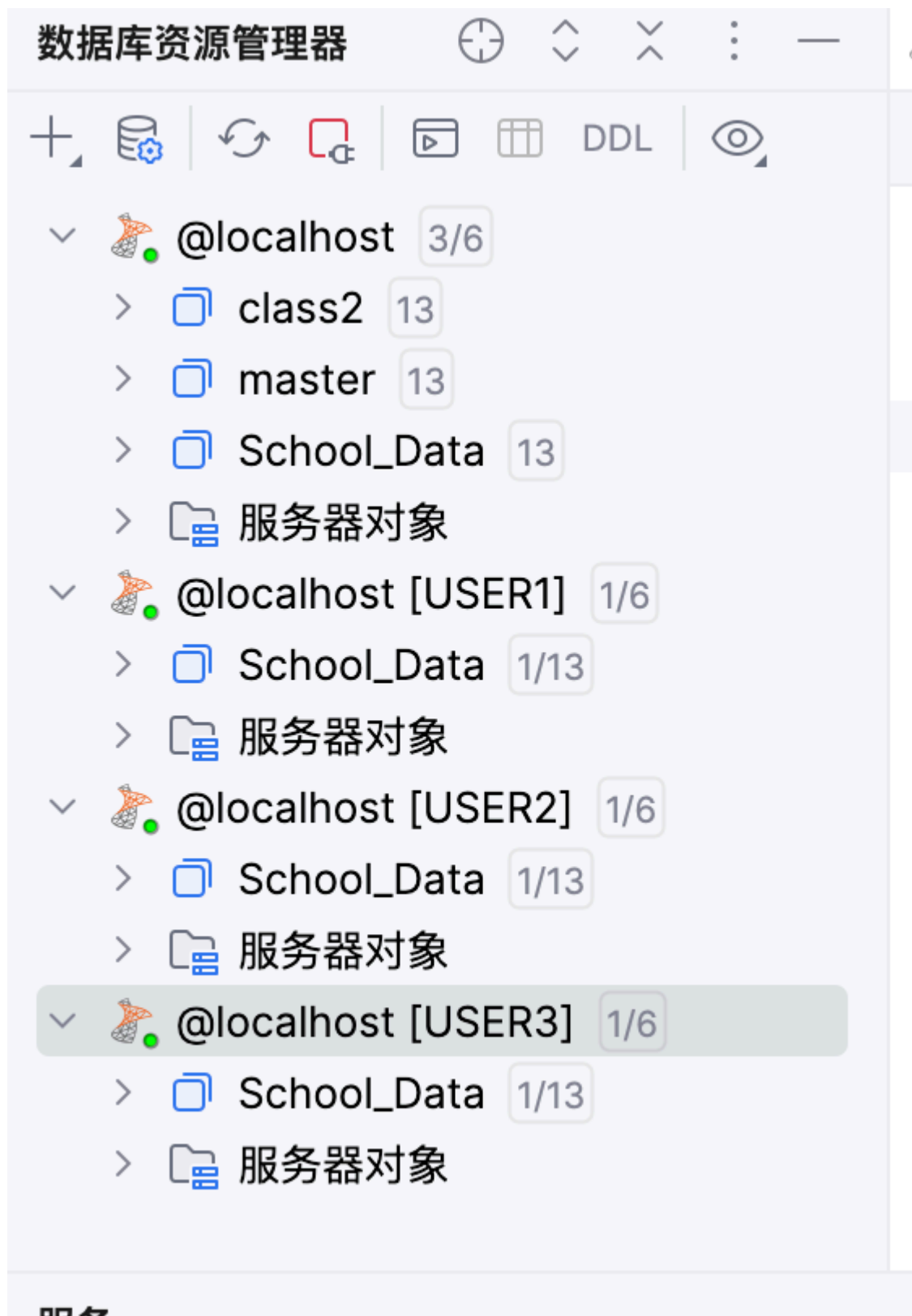
说明：本实验在SQL Server环境下进行，使用School_Data数据库，创建了三个测试用户USER1、USER2、USER3。

三、实验结果

环境准备

数据库用户架构

本实验涉及三个用户（USER1、USER2、USER3）和多个数据库对象（表和视图）。下图展示了实验中使用的数据库用户架构：



环境重置

首先重置实验环境，删除已存在的用户和登录名，然后重新创建：

-- 删除已存在的数据库用户

```
USE School_Data;
```

GO

```
IF EXISTS (SELECT * FROM sys.database_principals WHERE name = 'USER1') DROP USER USER1
```

```
IF EXISTS (SELECT * FROM sys.database_principals WHERE name = 'USER2') DROP USER USER2
```

```
IF EXISTS (SELECT * FROM sys.database_principals WHERE name = 'USER3') DROP USER USER3
```

GO

-- 删除已存在的登录名

```
USE master;
```

GO

```
IF EXISTS (SELECT * FROM sys.server_principals WHERE name = 'USER1') DROP LOGIN USER1;
```

```
IF EXISTS (SELECT * FROM sys.server_principals WHERE name = 'USER2') DROP LOGIN USER2;
```

```
IF EXISTS (SELECT * FROM sys.server_principals WHERE name = 'USER3') DROP LOGIN USER3;
```

GO

-- 重新创建登录名

```
CREATE LOGIN USER1 WITH PASSWORD = 'YourStrong!Passw0rd123', DEFAULT_DATABASE = School
```

```
CREATE LOGIN USER2 WITH PASSWORD = 'YourStrong!Passw0rd123', DEFAULT_DATABASE = School
```

```
CREATE LOGIN USER3 WITH PASSWORD = 'YourStrong!Passw0rd123', DEFAULT_DATABASE = School
```

GO

-- 重新创建数据库用户（无任何权限）

```
USE School_Data;
```

GO

```
CREATE USER USER1 FOR LOGIN USER1 WITH DEFAULT_SCHEMA = dbo;
```

```
CREATE USER USER2 FOR LOGIN USER2 WITH DEFAULT_SCHEMA = dbo;
```

```
CREATE USER USER3 FOR LOGIN USER3 WITH DEFAULT_SCHEMA = dbo;
```

GO

(1) 授予所有用户对表STUDENTS的查询权限

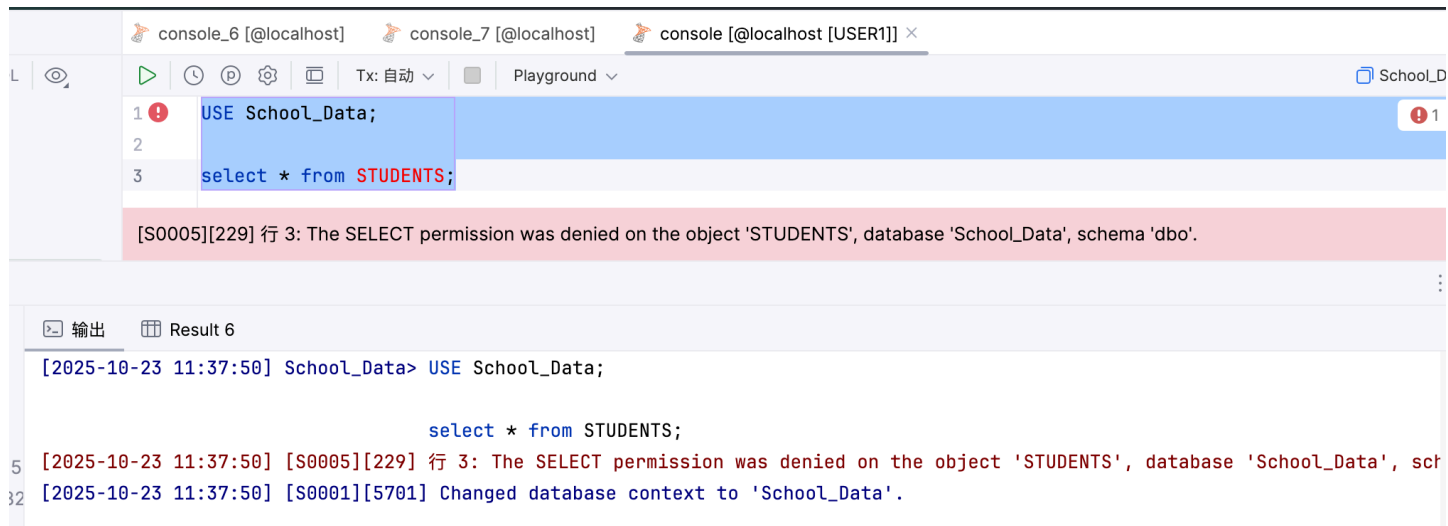
初始状态验证

在授权前，USER1尝试查询STUDENTS表会失败：

-- USER1执行

```
USE School_Data;
```

```
SELECT * FROM STUDENTS;
```



授权操作

使用PUBLIC关键字对所有用户授权：

```
-- 管理员执行
GRANT SELECT ON STUDENTS TO PUBLIC;
```

验证权限

```
SELECT dp.name AS 用户名, permission_name AS 权限类型, state_desc AS 状态
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE p.major_id = OBJECT_ID('STUDENTS')
AND dp.name IN ('USER1', 'USER2', 'USER3', 'public');
```


console_7 [localhost]

console [localhost [USER1]]

console [localhost [USER2]]

console [localhost

DL

Tx: 自动

Playground

1

USE School_Data;

2

3

select * from STUDENTS;

4

输出

Result 29

</

验证权限

```
SELECT dp.name AS 用户名, permission_name AS 权限类型, state_desc AS 状态
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE p.major_id = OBJECT_ID('COURSES')
AND dp.name IN ('USER1', 'USER2', 'USER3', 'public');
```

DDL

144 ✓
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160 ✓
161
162

```
SELECT
    dp.name AS 用户名,
    permission_name AS 权限类型,
    state_desc AS 状态
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE
    p.major_id = OBJECT_ID('COURSES')
    AND dp.name IN (
        'USER1',
        'USER2',
        'USER3',
        'public'
    );
GO

PRINT '';
GO
```

输出 Result 45

用户名 权限类型 状态

2	public	UPDATE	GRANT
3	USER1	SELECT	GRANT
4	USER1	UPDATE	GRANT
5	USER2	SELECT	GRANT
6	USER2	UPDATE	GRANT
7	USER3	SELECT	GRANT
8	USER3	UPDATE	GRANT

8行

说明：可以对同一对象授予多种权限，这里授予了SELECT和UPDATE两种权限。

(3) 授予USER1对表TEACHERS的查询、更新工资的权限，且允许传播

由于题目要求只授予对工资字段的权限，这里创建视图TS来实现列级权限控制：

```
-- 管理员执行
CREATE VIEW TS AS SELECT salary FROM TEACHERS;
GRANT SELECT ON TS TO USER1 WITH GRANT OPTION;
GRANT UPDATE ON TS TO USER1 WITH GRANT OPTION;
```

验证权限

```
SELECT dp.name AS 用户名, permission_name AS 权限类型, state_desc AS 状态,
       CASE WHEN p.state = 'W' THEN '是' ELSE '否' END AS 可传播
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE p.major_id = OBJECT_ID('TS')
AND dp.name = 'USER1';
```


@localhost console_7 [@localhost] x console [@localhost [USER1]] console [@localhost [USER2]] console

172 PRINT '验证: 查看USER1对TEACHERS表的权限';

173

174 ✓ SELECT

175 dp.name AS 用户名,

176 permission_name AS 权限类型,

177 state_desc AS 状态,

178 CASE

179 WHEN p.state = 'W' THEN '是'

180 ELSE '否'

181 END AS 可传播

182 FROM sys.database_permissions p

183 JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id

184 WHERE

185 p.major_id = OBJECT_ID('TS')

186 AND dp.name = 'USER1';

187 GO

188

189 ✓ PRINT '';

190 GO

191

💡 // ⌵ ⏮ ⏭ ⌵ ⋮

输出 Result 46 x

	用户名	权限类型	状态	可传播
1	USER1	SELECT	GRANT_WITH_GRANT_OPTION	是
2	USER1	UPDATE	GRANT_WITH_GRANT_OPTION	是

2 行

USER1测试权限

-- USER1执行

USE School_Data;

SELECT * FROM TEACHERS; -- 失败, 无权限

SELECT salary FROM TEACHERS; -- 失败, 无权限

SELECT salary FROM TS; -- 成功

```
console_7 [@localhost] x console [@localhost [USER1]] console [@localhost [USER2]] consol

DDL | 1/6 | 1/6 | 1/6 |

149 JOIN sys.database_principals up ON p.grantee_principal_id = up.principal_id
150 WHERE
151     p.major_id = OBJECT_ID('COURSES')
152     AND dp.name IN (
153         'USER1',
154         'USER2',
155         'USER3',
156         'public'
157     );
158 GO
159
160 PRINT '';
161 GO
162
163 -- (3)授予USER1对表 TEACHERS的查询,更新工资的权限,且允许USER1可以传播这些权限
164 PRINT '===== 题目(3): 授予USER1对表 TEACHERS的查询、更新工资的权限, 且允许传播 =
165
166 -- 在这里写你的代码
167 create view TS as select salary from TEACHERS;
168 ✓ grant select on TS to USER1 with grant option;
169 grant update on TS to USER1 with grant option;
170
171 -- 验证权限
```

```
输出 | Result 60
[2025-10-23 15:16:10] 在 5 ms 内完成
[2025-10-23 15:16:12] School_Data> grant update on TS to USER1 with grant option
[2025-10-23 15:16:12] 在 5 ms 内完成
[2025-10-23 15:16:20] School_Data> grant select on TS to USER1 with grant option;
                                grant update on TS to USER1 with grant option;
[2025-10-23 15:16:20] 在 6 ms 内完成
[2025-10-23 15:16:21] School_Data> grant select on TS to USER1 with grant option;
                                grant update on TS to USER1 with grant option;
[2025-10-23 15:16:21] 在 7 ms 内完成
```

说明:

- WITH GRANT OPTION允许USER1将该权限传播给其他用户
- 通过视图实现了列级权限控制, USER1只能访问salary字段
- state='W'表示该权限带有传播权 (WITH GRANT OPTION)

(4) 授予USER2对表CHOICES的查询、更新成绩的权限

同样使用视图实现列级权限控制:

-- 管理员执行

```
CREATE VIEW CS AS SELECT score FROM CHOICES;  
GRANT SELECT ON CS TO USER2;  
GRANT UPDATE ON CS TO USER2;
```

验证权限

```
SELECT dp.name AS 用户名, permission_name AS 权限类型, state_desc AS 状态  
FROM sys.database_permissions p  
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id  
WHERE p.major_id = OBJECT_ID('CS')  
AND dp.name = 'USER2';
```

@localhostconsole_7 [@localhost] ×console [@localhost [USER1]]console [@localhost [USER2]

197

198grant select on CS to USER2;

199grant update on CS to USER2;

200

201-- 验证权限

202✓PRINT '验证: 查看USER2对CHOICES表的权限';

203

204SELECT

205dp.name AS 用户名,

206permission_name AS 权限类型,

207state_desc AS 状态

208FROM sys.database_permissions p

209JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id

210WHERE

211p.major_id = OBJECT_ID('CS')

212AND dp.name = 'USER2';

213GO

214

215PRINT '';

输出Result 47 ×

用户名权限类型状态

1USER2SELECTGRANT

2USER2UPDATEGRANT

2行

USER2测试权限

```
-- USER2执行
USE School_Data;
SELECT score FROM CHOICES; -- 失败, 无权限
SELECT * FROM CS; -- 成功
```

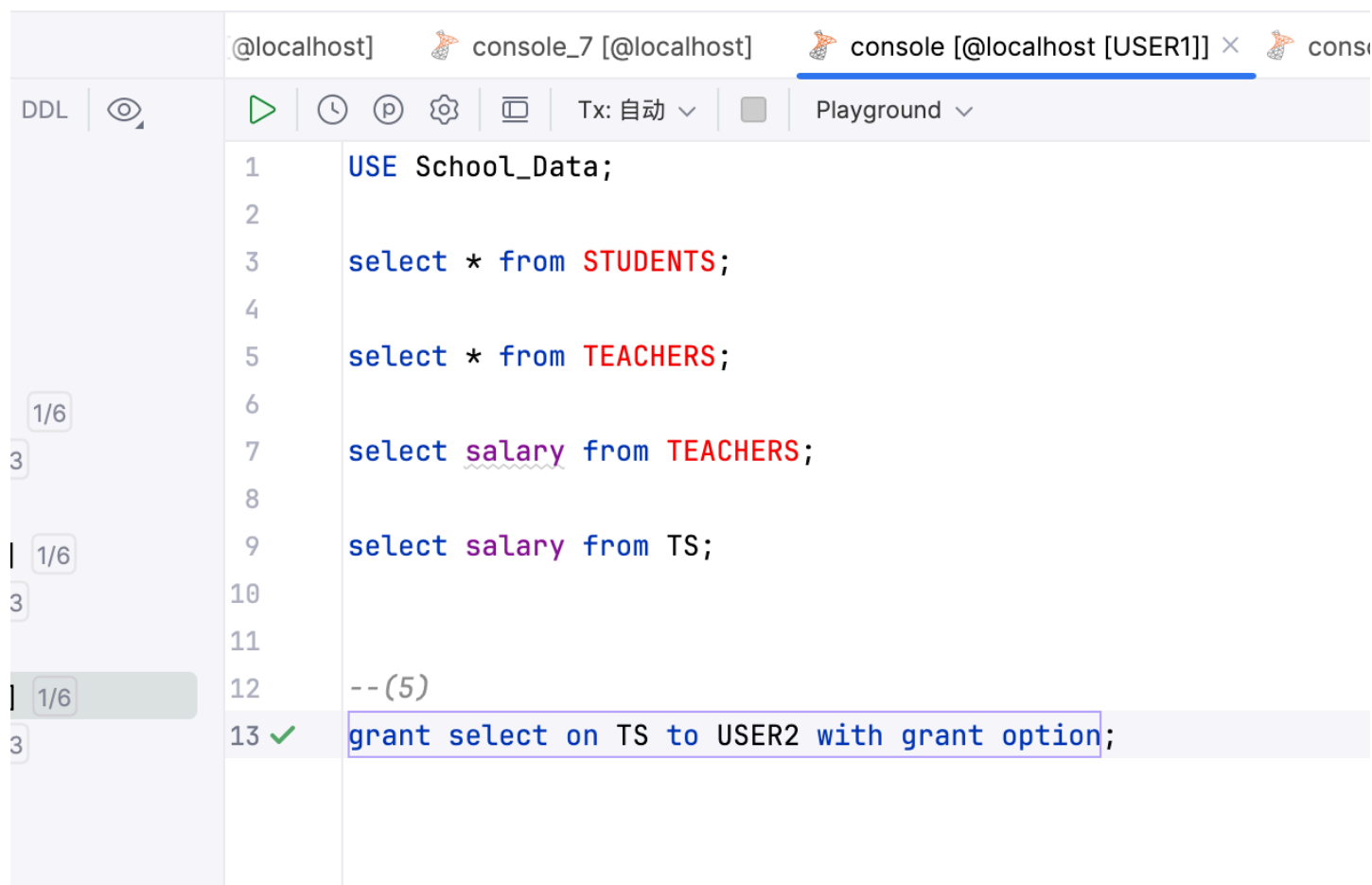
说明：USER2没有WITH GRANT OPTION，因此不能将权限传播给其他用户。

(5) 由USER1授予USER2对表TEACHERS的查询权限（带传播权）

由于USER1在题目(3)中获得了WITH GRANT OPTION，因此可以将权限传播给USER2：

-- USER1执行（在USER1的查询控制台）

```
GRANT SELECT ON TS TO USER2 WITH GRANT OPTION;
```



```
1  USE School_Data;
2
3  select * from STUDENTS;
4
5  select * from TEACHERS;
6
7  select salary from TEACHERS;
8
9  select salary from TS;
10
11
12  -- (5)
13  grant select on TS to USER2 with grant option;
```

验证权限

-- 管理员执行

```
SELECT dp.name AS 用户名, permission_name AS 权限类型, state_desc AS 状态,
       CASE WHEN p.state = 'W' THEN '是' ELSE '否' END AS 可传播
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE p.major_id = OBJECT_ID('TS')
AND dp.name = 'USER2';
```

console_7 [localhost] x console [localhost [USER1]] console [localhost [USER2]] console [localhost [USER3]]

DDL

3/6

ata 13

USER1] 1/6

ata 1/13

USER2] 1/6

ata 1/13

USER3] 1/6

ata 1/13

225 PRINT '验证: 查看USER2对TEACHERS表的权限';

226

227 ✓ SELECT

228 dp.name AS 用户名,

229 permission_name AS 权限类型,

230 state_desc AS 状态,

231 CASE

232 WHEN p.state = 'W' THEN '是'

233 ELSE '否'

234 END AS 可传播

235 FROM sys.database_permissions p

236 JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id

237 WHERE

238 p.major_id = OBJECT_ID('TS')

239 AND dp.name = 'USER2';

240 GO

241

242 PRINT '...'

输出 Result 73 x

用户名

权限类型

状态

可传播

1

USER2

SELECT

GRANT_WITH_GRANT_OPTION

是

1行

说明：USER1成功将权限传播给USER2，且USER2也获得了传播权。

(6) 测试权限传播链（循环授权）

USER2授予USER3权限

-- USER2执行（在USER2的查询控制台）

```
GRANT SELECT ON TS TO USER3 WITH GRANT OPTION;
```

```
@localhost console_7 [@localhost] console [@localhost [USER1]] console [@localhost [USER2]] x
隐藏 ↑ ↺
1 use School_Data;
2
3 select score from CHOICES;
4
5 select * from CS;
6
7 --(6)
8 ✓ grant select on TS to USER3 with grant option;
```

USER3授予USER2权限

-- USER3执行（在USER3的查询控制台）

```
GRANT SELECT ON TS TO USER2 WITH GRANT OPTION;
```

```
@localhost console_7 [@localhost] console [@localhost [USER1]] console [@localhost [USER2]] console [@localhost [USER3]] x
1 use School_Data;
2
3 --(6)
4 ✓ grant select on TS to USER2 with grant option;
```

验证权限状态

-- 管理员执行

```
SELECT dp.name AS 用户名, permission_name AS 权限类型, state_desc AS 状态,
       CASE WHEN p.state = 'W' THEN '是' ELSE '否' END AS 可传播
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE p.major_id = OBJECT_ID('TS')
AND dp.name IN ('USER2', 'USER3');
```


- 管理员授予USER1权限（带传播权）
- USER1授予USER2权限（带传播权）
- USER2授予USER3权限（带传播权）
- USER3又授予USER2权限（带传播权，形成循环）

-- 管理员执行

```
SELECT dp.name AS 用户名, permission_name AS 权限类型, state_desc AS 状态,
       CASE WHEN p.state = 'W' THEN '是' ELSE '否' END AS 可传播
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE p.major_id = OBJECT_ID('TS')
AND dp.name IN ('USER1', 'USER2', 'USER3');
```

```
281 ✓ SELECT
282     dp.name AS 用户名,
283     permission_name AS 权限类型,
284     state_desc AS 状态
285 FROM sys.database_permissions p
286 JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
287 WHERE
288     p.major_id = OBJECT_ID('TS')
289     AND dp.name IN ('USER1', 'USER2', 'USER3','public');
290 GO
291
292 PRINT '';
293 GO
294
```

用户名	权限类型	状态	可传播
public	SELECT	GRANT_WITH_GRANT_OPTION	
public	UPDATE	GRANT_WITH_GRANT_OPTION	
USER1	SELECT	GRANT_WITH_GRANT_OPTION	
USER1	UPDATE	GRANT_WITH_GRANT_OPTION	
USER2	SELECT	GRANT_WITH_GRANT_OPTION	
USER2	SELECT	GRANT_WITH_GRANT_OPTION	
USER3	SELECT	GRANT_WITH_GRANT_OPTION	

可以看到，取消前USER1、USER2、USER3都拥有对TS的查询权限，其中USER2有两条记录（分别来自USER1和USER3的授权）。

执行取消操作

-- 管理员执行

```
REVOKE SELECT ON TS FROM USER1 CASCADE;
```

3/6

3

13

Data 13

象

[USER1] 1/6

Data 1/13

象

[USER2] 1/6

Data 1/13

象

[USER3] 1/6

Data 1/13

象

@localhost console_7 [@localhost] x console [@localhost [USER1]] console [@localhost [USER2]]

Tx: 自动 Playground

253 PRINT '验证: 查看USER3对TEACHERS表的权限';

254

255 SELECT

256 dp.name AS 用户名,

257 permission_name AS 权限类型,

258 state_desc AS 状态,

259 CASE

260 WHEN p.state = 'W' THEN '是'

261 ELSE '否'

262 END AS 可传播

263 FROM sys.database_permissions p

264 JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id

265 WHERE

266 p.major_id = OBJECT_ID('TS')

267 AND dp.name IN ('USER2', 'USER3');

268 GO

269

270 PRINT '';

271 GO

272

273 -- (7)取消USER1对表 TEACHERS的查询权限,考虑由USER2的身份对表 TEACHERS进行查询,操作能否成功?

274 PRINT '===== 题目(7): 取消USER1对表 TEACHERS的查询权限, 测试USER2 =====';

275

276 -- 在这里写你的代码

277 ✓ revoke select on TS from USER1 cascade;

278

279 -- 验证权限

输出 Result 52

p.major_id = OBJECT_ID('TS')

AND dp.name IN ('USER1', 'USER2', 'USER3','public')

[2025-10-23 14:49:06] 在 347 ms (execution: 28 ms, fetching: 319 ms) 内检索到从 1 开始的 7 行

[2025-10-23 14:51:41] School_Data> revoke select on TS from USER1 cascade

[2025-10-23 14:51:41] 在 15 ms 内完成

取消后查看权限

```
SELECT dp.name AS 用户名, permission_name AS 权限类型, state_desc AS 状态
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE p.major_id = OBJECT_ID('TS')
AND dp.name IN ('USER1', 'USER2', 'USER3');
```

取消后-权限状态

可以看到，取消后所有用户的权限记录都消失了，包括USER3授予给USER2的权限也被级联撤销。

验证用户无法访问

```
-- USER2执行
SELECT * FROM TS; -- 失败，权限不足
```

```
-- USER3执行
SELECT * FROM TS; -- 失败，权限不足
```

所有用户现在都无法查询TS视图。

重要发现与讨论：

1. **CASCADE的强大威力**：使用 REVOKE ... CASCADE 会级联撤销所有由该用户直接或间接传播出去的权限，即使存在循环授权也不例外。
2. **权限传播链的完全断裂**：权限传播链为：管理员→USER1→USER2→USER3→USER2（循环）。当撤销USER1的权限时，整个链条完全断裂，所有下游用户的权限都被撤销。
3. **循环授权也会被级联撤销**：虽然USER3授予了USER2权限（形成循环），但这个权限的根源仍然是管理员通过USER1传播的。一旦撤销USER1的权限，即使是循环授权的权限也会被级联撤销。这说明SQL Server能够追踪权限的授权来源，并在撤销时进行完整的依赖分析。
4. **防止孤儿权限**：CASCADE机制确保了权限的一致性，防止出现“孤儿权限”（即授权者已失去权限，但被授权者仍保留权限的情况）。这是数据库安全的重要保障。
5. **实际应用中的注意事项**：在生产环境中使用CASCADE撤销权限时必须特别小心，因为一次操作可能影响整个权限传播链上的所有用户，可能导致大量用户突然失去访问权限。建议在执行前先查询权限依赖关系，评估影响范围。

(8) 取消USER1和USER2关于表COURSES的权限

取消前查看权限

```
-- 管理员执行
SELECT dp.name AS 用户名, permission_name AS 权限类型, state_desc AS 状态
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE p.major_id = OBJECT_ID('COURSES')
AND dp.name IN ('USER1', 'USER2', 'public');
```

DDL

console_7 [localhost] x console [localhost [USER1]] console [localhost [USER2]] console [localhost [USER3]]

299 -- 在这里写你的代码
300 revoke select on COURSES from USER1;
301 revoke update on COURSES from USER1;
302 revoke select on COURSES from USER2;
303 revoke update on COURSES from USER2;
304
305 -- 验证权限
306 PRINT '验证: 查看COURSES表的权限';
307
308 ✓ SELECT
309 dp.name AS 用户名,
310 permission_name AS 权限类型,
311 state_desc AS 状态
312 FROM sys.database_permissions p
313 JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
314 WHERE
315 p.major_id = OBJECT_ID('COURSES')
316 AND dp.name IN ('USER1', 'USER2', 'public');
317 GO
318
319 PRINT '';

输出 Result 55 x

用户名	权限类型	状态
1 public	SELECT	GRANT
2 public	UPDATE	GRANT
3 USER1	SELECT	GRANT
4 USER1	UPDATE	GRANT
5 USER2	SELECT	GRANT
6 USER2	UPDATE	GRANT

执行取消操作

```
-- 管理员执行
REVOKE SELECT ON COURSES FROM USER1;
REVOKE UPDATE ON COURSES FROM USER1;
REVOKE SELECT ON COURSES FROM USER2;
REVOKE UPDATE ON COURSES FROM USER2;
```

DDL

console_7 [localhost] x console [localhost [USER1]] console [localhost [USER2]] console [localhost [USER3]]

Tx: 自动 Playground

287 JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id

288 WHERE

289 p.major_id = OBJECT_ID('TS')

290 AND dp.name IN ('USER1', 'USER2', 'USER3', 'public');

291 GO

292

293 PRINT '';

294 GO

295

296 -- (8)取消USER1和USER2的关于表 COURSES的权限

297 PRINT '===== 题目(8): 取消USER1和USER2关于表 COURSES的权限 =====';

298

299 -- 在这里写你的代码

300 ✓ revoke select on COURSES from USER1;

301 revoke update on COURSES from USER1;

302 revoke select on COURSES from USER2;

303 revoke update on COURSES from USER2;

304

305 -- 验证权限

306 PRINT '验证: 查看COURSES表的权限';

307

Unicode 字符串应具有 N 前缀

添加 N 前缀 更多操作...

输出 Result 55

JOIN sys.database_principals dp ON p.grantee_principal_id =

WHERE

p.major_id = OBJECT_ID('COURSES')

AND dp.name IN ('USER1', 'USER2', 'public')

[2025-10-23 14:56:11] 在 349 ms (execution: 32 ms, fetching: 317 ms) 内检索到从 1 开始的 6 行

[2025-10-23 14:56:44] School_Data> revoke select on COURSES from USER1;

revoke update on COURSES from USER1;

revoke select on COURSES from USER2;

revoke update on COURSES from USER2;

[2025-10-23 14:56:44] 在 12 ms 内完成

取消后查看权限

```
SELECT dp.name AS 用户名, permission_name AS 权限类型, state_desc AS 状态
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
WHERE p.major_id = OBJECT_ID('COURSES')
AND dp.name IN ('USER1', 'USER2', 'public');
```

console_7 [localhost] x console [localhost [USER1]] console [localhost [USER2]] cc

300

301

302

303

304

305

306

307

308 ✓

309

310

311

312

313

314

315

316

317

318

319

320

```
revoke select on COURSES from USER1;
revoke update on COURSES from USER1;
revoke select on COURSES from USER2;
revoke update on COURSES from USER2;

-- 验证权限
PRINT '验证: 查看COURSES表的权限';

SELECT
    dp.name AS 用户名,
    permission_name AS 权限类型,
    state_desc AS 状态
FROM sys.database_permissions p
JOIN sys.database_principals dp ON p.grantee_principal_id = dp.princip
WHERE
    p.major_id = OBJECT_ID('COURSES')
    AND dp.name IN ('USER1', 'USER2', 'public');

GO

PRINT '';
```

输出 Result 57 x

用户名 权限类型 状态

1 public SELECT GRANT

2 public UPDATE GRANT

2行

说明：由于USER1和USER2的权限来自PUBLIC，直接REVOKE USER1和USER2不会生效，因为他们仍然通过PUBLIC角色拥有权限。

取消PUBLIC的权限

一、管理员执行

REVOKE SELECT ON COURSES FROM PUBLIC;

REVOKE UPDATE ON COURSES FROM PUBLIC;

The screenshot displays a SQL development environment with a query editor and a results pane.

Query Editor:

```
306 ✓ REVOKE select on COURSES from public;
307 REVOKE update on COURSES from public;
308
309 -- 验证权限
310 PRINT '验证: 查看COURSES表的权限';
311
312 SELECT
313     dp.name AS 用户名,
314     permission_name AS 权限类型,
315     state_desc AS 状态
316 FROM sys.database_permissions p
317     JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
318 WHERE
319     p.major_id = OBJECT_ID('COURSES')
320     AND dp.name IN ('USER1', 'USER2', 'public');
321 GO
322
323 PRINT '验证完成';
```

Results Pane:

The results pane shows the output of the query, titled "输出 | 计划" (Output | Plan) and "Result 86". The results are displayed in a table with columns: 用户名 (Username), 权限类型 (Permission Type), and 状态 (State).

用户名	权限类型	状态
-----	------	----

The results pane indicates 0 rows of data.

验证USER1和USER2无法访问

-- USER1执行

```
SELECT * FROM COURSES;
```

The screenshot shows a SQL Server console window with four tabs: console_7 [localhost], console [localhost [USER1]], console [localhost [USER2]], and console [localhost [USER3]]. The active tab is console [localhost [USER1]]. The SQL script in the editor is as follows:

```
2  
3  select * from STUDENTS;  
4  
5  select * from TEACHERS;  
6  
7  select salary from TEACHERS;  
8  
9  select salary from TS;  
10  
11  
12  --(5)  
13  grant select on TS to USER2 with grant option;  
14  
15  -- (8)  
16  select * from COURSES;
```

Below the script, a red error bar displays the message: [S0005][229] 行 1: The SELECT permission was denied on the object 'COURSES', database 'School_Data', schema 'dbo'.

-- USER2执行

```
SELECT * FROM COURSES;
```

The screenshot shows a SQL Server console window with four tabs: console_7 [localhost], console [localhost [USER1]], console [localhost [USER2]], and console [localhost [USER3]]. The active tab is console [localhost [USER2]]. The SQL script in the editor is as follows:

```
1  use School_Data;  
2  
3  select score from CHOICES;  
4  
5  select * from CS;  
6  
7  --(6)  
8  grant select on TS to USER3 with grant option;  
9  
10  --(7)  
11  select * from TS;  
12  
13  --(8)  
14  select * from COURSES;
```

Below the script, a red error bar displays the message: [S0005][229] 行 1: The SELECT permission was denied on the object 'COURSES', database 'School_Data', schema 'dbo'.

说明：

- 通过PUBLIC授予的权限需要从PUBLIC撤销才能完全取消
- 单独撤销某个用户的权限不会影响该用户通过PUBLIC获得的权限
- 撤销PUBLIC的权限后，所有用户（包括USER1、USER2、USER3）都无法访问COURSES表

四、问题&总结

通过本次实验，我深入理解了SQL Server的权限管理机制，包括授权、权限传播和权限撤销等核心概念。

关于授权机制：SQL Server提供了灵活的授权方式。使用PUBLIC关键字可以一次性授予所有用户权限，这在需要公开某些数据时非常方便。但PUBLIC授权也有风险，因为它会影响所有用户，包括未来创建的用户。对于需要精细控制的场景，应该针对特定用户或角色授权。

关于列级权限控制：SQL Server不直接支持列级GRANT语句，但可以通过创建视图来实现列级权限控制。在题目(3)和(4)中，我创建了TS和CS视图，分别只包含TEACHERS表的salary字段和CHOICES表的score字段，然后对视图授权。这样用户只能访问特定的列，而不能访问整个表。这种方法在实际应用中很有用，可以保护敏感数据。

关于权限传播 (WITH GRANT OPTION)：WITH GRANT OPTION是一个强大但需要谨慎使用的特性。它允许被授权者将权限继续传播给其他用户，形成权限传播链。在题目(5)中，USER1将权限传播给USER2，USER2又传播给USER3。这种机制在分级管理中很有用，但也可能导致权限失控。

关于循环授权：题目(6)测试了一个有趣的场景：USER2授权给USER3，USER3又授权给USER2，形成循环。我发现SQL Server允许这种循环授权，不会报错。系统视图中甚至出现了两条USER2的记录，因为USER2从两个不同的授权者（USER1和USER3）获得了权限。这说明SQL Server的权限系统记录的是每一次授权操作，而不是简单的"有权限"或"无权限"的状态。虽然循环授权在技术上可行，但在权限管理上可能造成混乱，应该避免。

关于权限撤销的级联效应：题目(7)展示了REVOKE CASCADE的强大威力和精确的依赖追踪能力。当撤销USER1的权限时，所有由USER1直接或间接传播出去的权限都会被级联撤销。权限传播链是：管理员→USER1→USER2→USER3→USER2（循环），撤销USER1的权限会导致整个链条完全断裂。最令人印象深刻的是，即使USER3授予了USER2权限（形成循环授权），这个权限也会被级联撤销，因为SQL Server能够追踪到这个权限的根源仍然是通过USER1传播的。实验结果显示，撤销后USER1、USER2、USER3都完全失去了对TS的访问权限，没有任何"孤儿权限"残留。这个机制确保了权限的一致性和安全性，但在实际应用中需要特别小心，因为一次撤销操作可能影响整个权限传播链上的所有用户，造成大范围的访问中断。

关于PUBLIC角色的特殊性：题目(8)让我认识到PUBLIC角色的特殊性。在题目(2)中，我通过PUBLIC授予了所有用户对COURSES表的权限。在题目(8)中，当我尝试撤销USER1和USER2的权限时，发现单独撤销某个用户不起作用，因为他们仍然通过PUBLIC角色拥有权限。必须撤销PUBLIC的权限，才能真正取消所有用户的访问权限。这说明PUBLIC是一个特殊的角色，所有用户都自动属于PUBLIC，通过PUBLIC授予的权限需要从PUBLIC撤销。

实验中的技术细节：在实验过程中，我采用了在不同数据源创建查询控制台的方式来模拟多用户环境。这种方法比使用EXECUTE AS USER更直观，可以真实地测试不同用户的权限。每个用户都有独立的连接和会话，更接近实际的多用户场景。

权限管理的最佳实践：通过这次实验，我总结了一些权限管理的最佳实践：

1. 遵循最小权限原则，只授予必要的权限
2. 谨慎使用WITH GRANT OPTION，避免权限失控
3. 避免过度依赖PUBLIC角色，对敏感数据使用特定用户授权
4. 使用视图实现列级权限控制和行级权限控制
5. 定期审查权限，及时撤销不再需要的权限
6. 使用CASCADE撤销权限时要特别小心，因为会影响权限传播链上的所有用户

系统视图的使用：实验中大量使用了sys.database_permissions和sys.database_principals等系统视图来查看权限状态。这些视图提供了权限管理的透明度，可以清楚地看到谁拥有什么权限、权限是否可传播、权限的授予者是谁等信息。掌握这些系统视图的使用对于权限管理和故障排查非常重要。

总的来说，这次实验让我对数据库的安全机制有了更深入的理解。权限管理是数据库安全的核心，正确使用GRANT和REVOKE语句，合理设计权限结构，对于保护数据安全和维护系统稳定至关重要。